CSC 288.2

Interactive Programming Using JavaScript

A comprehensive study guide compiled from lecture notes, covering interactive programming concepts, DOM manipulation, conditional logic, and loops — with a Java-based interactive programming comparison.

Topics covered:

1. What Is Interactive Programming?

Definition: Programming is called *interactive* when you can carry out all operations — write, execute, modify, and read — in real time, from start to end, without restarting the whole program.

Because operations happen live, you get **immediate feedback** without waiting for the program to fully recompile. This concept applies broadly across software development, and is often called **live coding** or the **REPL** model:

> **REPL** = Read → Eval → Print → Loop

In interactive/live coding, the developer **communicates directly with the software** as it runs, rather than writing a complete program, saving it, compiling it, and only then seeing the result.

Key Activity: The Interactive Process

Instead of writing a full file, saving, compiling, and running it, you type a single command — the system processes it and returns output **immediately**.

> A program built this way will always have both its **instructions (program/code)** and its **inputs (data)** present at the same time — this is why *"Instruction and Data go hand in hand."*

Advantages of Interactive Programming

- Allows programmers to experiment, test ideas, and visualize data on the fly, without stopping the program.
- Enables faster learning than any other method of programming.
- Gives immediate feedback — errors and results are seen instantly.

2. Java Interactive Example (Scanner & Random)

Even in a compiled language like Java, you can build an interactive program by combining **Scanner** (to read live user input) with **Random** (to generate data on the fly) — this keeps instructions and data flowing together at runtime, e.g. in a simple guessing game.

```
import java.util.Scanner;
import java.util.Random;

public class InteractiveGame {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Random random = new Random();

        int secretNumber = random.nextInt(100) + 1;  // random 1-100
        int guess = 0;
        int attempts = 0;

        System.out.println("Guess the number between 1 and 100!");

        while (guess != secretNumber) {
```

```
            System.out.print("Enter your guess: ");
            guess = scanner.nextInt();
            attempts++;

            if (guess > secretNumber) {
                System.out.println("Too high! Try again.");
            } else if (guess < secretNumber) {
                System.out.println("Too low! Try again.");
            } else {
                System.out.println("Correct! You got it in " + attempts + " attempts.");
            }
        }

        scanner.close();
    }
}
```

How it works: Scanner reads live input from the keyboard (interactive input), and **Random** generates the secret number on the fly (interactive data) — the **while** loop keeps the program running, reading a new guess and giving feedback, until the user's guess matches the secret number. This is a compiled language achieving the same "instruction and data go hand in hand" idea your notes describe for REPL-style languages.

3. The DOM (Document Object Model)

The DOM represents an HTML page as a **hierarchical tree structure**, allowing JavaScript to dynamically access, modify, and style its contents. As a browser loads a page, it converts the raw HTML text into a **tree of nodes** that can be targeted using standard JS commands.

4. Selecting Elements

Command	What it does
document.querySelector('selector')	Selects the first single element matching the given CSS selector.
document.querySelectorAll('selector')	Selects ALL elements matching the given selector (returns a list/NodeList).
document.getElementById('id')	Selects a single element by its specific id attribute.
document.getElementsByClassName('className')	Returns a live HTMLCollection of all elements with that class name.

5. Output Commands

Command	What it does
console.log()	An output command usable on both the client (browser) and server side (Node.js).
process.stdout.write()	Prints text directly to the terminal/system without introducing any line break.

Difference in a nutshell: **console.log()** auto-formats and adds a newline after each call; **process.stdout.write()** writes raw text with no automatic line break.

6. Modifying Content & Attributes

Once you've selected an element, you can change/update what it displays, or change its underlying HTML attributes:

Command	What it does
element.textContent	Alters plain text inside an element (no HTML parsing).
element.innerHTML	Sets text including raw HTML tags — the browser parses them into real elements.
element.setAttribute('attr', 'val')	Sets a specific HTML attribute (e.g. src, href, id) on the element.

7. Creating & Destroying Elements

Command	What it does
document.createElement('tag')	Creates a brand-new (unattached) HTML element in memory.
parent.appendChild(childNode)	Attaches the child element as the last child of the parent.
parent.removeChild(childNode)	Removes a specific child element from its parent.
element.remove()	Removes the element itself directly from the DOM.

8. Working with classList

The **classList** property gives access to an element's CSS classes, letting you add or remove classes to control styling dynamically:

```
element.classList.add('className')     // Adds a class
element.classList.remove('className')  // Removes a class
element.classList.toggle('className')  // Adds it if absent, removes it if present
```

toggle() is especially useful for on/off UI states — e.g. showing/hiding a menu or switching a dark-mode class.

9. Conditional Statements

A conditional statement lets you control the flow of a program by executing certain blocks of code based on whether a condition evaluates to **true** or **false**. Conditional operators evaluate to true or false, adding logic in your code and helping you build interactivity.

The if Statement

Runs a block of code only when its condition is true.

```
let rightAnswer = 40;
if (myScore > 75) {
    console.log("Total awesome!");
}
```

The if...else Statement

Runs one block if the condition is true, and a **different** block if it is false — a program can branch between exactly two outcomes depending on whether the condition is met.

```
let age = 10;
if (age >= 18) {
    console.log("You are eligible to vote");
} else {
    console.log("You cannot vote");
}
```

The if...else if...else Statement

Used when choosing between exactly two outcomes isn't enough — this runs specific blocks of code only when their own condition is met.

```
let gpa = 4.5;
if (gpa >= 4.5) {
    console.log("First Class");
} else if (gpa >= 3.5) {
    console.log("Second Class Upper");
} else {
    console.log("Keep working!");
}
```

10. Comparison & Logical Operators

A **comparison operator** is a compact way of comparing two values, evaluating conditions in a data structure to determine an outcome. It compares its two operands and returns true or false; only operators in a data structure that take two operands are called binary operators.

Operator	Meaning	Example
===	Strict equal to	if (score === 10)

!==	Not equal to	if (grade !== 'A')
>=	Greater than or equal to	if (age >= 18)
<=	Less than or equal to	if (age <= 15)
&&	AND — both conditions must be true	if (age >= 18 && score >= 60)
||	OR — at least one condition must be true	if (age < 18 || youCantVote)

11. Loops

The aim of a loop in a data structure is to repeat a block of code while recurring variables get used/updated. A loop will keep running until a determined condition is met or until a set number of iterations completes.

The for Loop

Starts by initializing **i = 0**, evaluates a condition **before** each pass, and runs until the condition becomes false.

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

The while Loop

Evaluates a condition **before** executing — starts running as long as the condition remains true.

```
let i = 0;
while (i < 5) {
    console.log(i);
    i++;
}
```

The do...while Loop

Runs the code block first, then evaluates the condition **after** execution — so the block always executes at least once, even if the condition is false from the start.

```
let i = 0;
do {
    console.log(i);
    i++;
} while (i < 5);
```

break

Used inside a loop to stop it immediately, regardless of the loop's own condition:

```
for (let i = 0; i < 10; i++) {
    if (i === 5) {
        break;
    }
```

```
    console.log(i);
}
```

Loop Cheat-Sheet

Loop	Condition checked	Guarantees at least 1 run?
for	Before each pass	No
while	Before each pass	No
do...while	After each pass	Yes

End of guide. Compiled from handwritten lecture notes for CSC 288.2 — ready for review before your next CBT or lab session.